

Bias Analysis for Theoretical Physics

Fernando Morales

December 16, 2025

1 Problem Statement

Literature review — especially in theoretical physics — is time-consuming, repetitive, and prone to structural bias: it is hard to quickly identify echo chambers, topic homophily, and epistemic tone that can reinforce paradigmatic lock-in. My motivation comes from a parallel pain point in political news consumption, where left/right media ecosystems and outlet-level incentives make it difficult to stay broadly informed without being pulled into a single narrative. In academia, a similar dynamic appears at scale: thousands of new papers arrive constantly, many low-signal or effectively “spam,” and entire subfields can become saturated with near-duplicate ideas while less orthodox but potentially valuable directions are ignored. I therefore built an agent/app that automates arXiv discovery, semantic clustering, and interactive visualization of research streams, estimates lightweight epistemic-stance signals from abstracts, constructs similarity networks to reveal clusters and missing bridges, and uses LLM-assisted comparison to help me decide what to read, what to deprioritize, and how scientific attention—and bias—shifts over time.

2 Prototype

The prototype is a **Streamlit** application with four tabs:

- **Search:** queries arXiv, downloads metadata (title, abstract, year, URL), and builds the working paper set.
- **Bias Analysis:** projects embeddings to 2D (PCA) and plots an interactive scatter (Plotly); point size reflects the magnitude of the epistemic score.
- **Networks:** builds a cosine-similarity graph (NetworkX), detects communities (Louvain), and renders an interactive network (PyVis).
- **Chat:** uses an LLM (Gemini via API) to compare/contrast retrieved papers and answer research questions using the retrieved context.

3 Technical architecture

3.1 Data flow

1. **Ingestion:** arXiv API $\rightarrow$ list of documents with `title`, `abstract`, `year`, `url`.
2. **Semantic representation:** dense embeddings with `sentence-transformers` (model `all-MiniLM-L6-v2`).
3. **Clustering:** `KMeans` for fast topical clustering (up to 4 clusters, or fewer for small n).
4. **Epistemic stance score:** lexicon-based counting of *hedges* and *boosters* over abstract lemmas (`spaCy`).

5. **Network:** edges created when $\cos(\mathbf{e}_i, \mathbf{e}_j) > \tau$ (threshold), weighted by similarity.
6. **Interaction:** visualizations (Plotly/PyVis) + LLM.

4 Implementation

The initial problem definition evolved quite a bit throughout the process. I had multiple ideas in mind, but this one was the most concrete in terms of how to execute it. Still, that didn't make it easy. One of the first challenges I faced was deciding how to approach the problem.

To approach the problem, I started by identifying epistemic bias (how speculative or assertive a text is and which metrics would help to determine it), and contrasting methodologies across multiple sources.

Also, I considered a large-scale solution—pulling and analyzing thousands of academic papers from multiple databases. However, I soon realized that this wouldn't be practical for real-world use. After all, as a researcher, I'm not going to read thousands of papers—I'm going to focus on a handful of the most relevant ones. That's when I decided to focus the system on retrieving a smaller, higher-quality set of papers—20, for example—using sources like arXiv and Semantic Scholar, also it means less computing time and less resources.

This decision also made it easier to classify the content and make better use of clustering and semantic similarity. A key idea I had early on was to analyze epistemic bias—but figuring out how to quantify it was a challenge. I wanted to be able to say, "this paper sounds speculative, while that one sounds assertive," and needed a metric to support that claim. All these things required going back to techniques I hadn't used in a while, like linguistic feature extraction with spaCy, and clustering using KMeans.

Clustering was especially useful in establishing groups of papers and mapping thematic similarity. It helped me build semantic proximity networks and visualize research positions. Additionally, designing and tuning the epistemic score based on linguistic markers (like hedges and boosters) required iteration and experimentation.

When it came to implementing AI-generated responses, I initially considered using GPT models. However, since GPT-4 access is paid, I looked for a more accessible alternative and ultimately integrated Gemini by Google. I evaluated it and found it suitable for contrastive analysis between papers. The idea was to let the user upload PDFs, extract their content (using OCR if necessary), and ask natural language questions like "How do these papers differ in methodology?"

Throughout the project, I faced technical and conceptual challenges which made time management a real concern. Completing the project under a tight deadline pushed me to think and act more like I would in a real-world research or product environment.

Ultimately, the project became a mix of concepts and tools. It was a rewarding experience—one that helped me consolidate even more theoretical knowledge.

5 How I used AI tools during development

Colab coding copilot for identifying errors, provided visualization templates and comment the final work.